

A
Project report on Course Program I
on

Multiplayer Bingo-Game

A Real-Time Cross-Platform Mobile Gaming Application built with
MERN and Flutter

Submitted in Partial Fulfillment of the Requirement for the Award of the Degree of
Bachelor of Computer Application

Submitted by

Aditya Jaiswal

Enrollment Number: 24TSBCDDD01034

BCA – 3rd Year (Batch 2024–2027)



Under the guidance of

Mr. Divya Shikar

(Designation – ICFAI Tech School)

Course Reference: Udemy – “Flutter & Dart – The Complete Guide”



ICFAI TECH SCHOOL
THE ICFAI UNIVERSITY, DEHRADUN

August – 2026

**ICFAI TECH SCHOOL
THE ICFAI UNIVERSITY, DEHRADUN**

CERTIFICATE

This is to certify that the Course Program I project entitled “**Multiplayer Bingo-Game – A Real-Time Cross-Platform Gaming Application**” has been carried out by **Mr. Aditya Jaiswal** (Enrollment No. 24TSBCDDD01034), a student of **Bachelor of Computer Application** {Batch 2024–2027}, during the Course Program I which was undertaken through self-paced structured learning on the Udemy platform under the course “**Flutter & Dart – The Complete Guide**”, commenced for the period of May to July of the year 2026.

It is certified that all corrections and suggestions indicated have been incorporated in the report. The project has been approved as it satisfies the academic requirements in respect of the Course Program I.

Signature of IP Faculty

ACKNOWLEDGEMENT

I would like to take this opportunity to express my sincere gratitude to everyone who supported me during the completion of my Course Program I.

I am thankful to my faculty guide, **Divya Shikar**, and the Course Program coordination team at ICFAI Tech School, The ICFAI University, Dehradun, for their constant guidance, valuable suggestions, and encouragement throughout the duration of this project. Their feedback on the architecture and implementation of the **Multiplayer Bingo-Game** helped me structure my learning and my work in a professional, industry-aligned manner.

I am also grateful to Udemy and the instructors of the “**Flutter & Dart Programming**” course, whose structured video lectures, hands-on assignments, and project-based teaching approach gave me the foundational and applied knowledge of Flutter, Dart, and mobile application architecture that I used to design and build this project.

I would like to thank my family and friends for their continuous motivation and patience during the long hours of coding, debugging, and testing that went into building a real-time multiplayer system. Finally, I thank the open-source community — the maintainers of Flutter, Node.js, Express, Socket.IO, and MongoDB — whose documentation and libraries made this project possible.

This Course experience has been an invaluable step in my journey towards becoming an **Software Engineer** (Web & App Developer), and I am thankful for every person who, directly or indirectly, contributed to it.

Aditya Jaiswal

Enrollment No. 24TSBCDDD01034

ABSTRACT

This report documents the work carried out during Course Program I of the Bachelor of Computer Applications (BCA) programme, undertaken between May and July 2026. The course was completed through structured self-paced learning on the Udemy platform, specifically the “Flutter & Dart Programming” course, which was used as the technical foundation for building a substantial, production-style software project rather than a set of isolated exercises.

The chosen project, titled “Multiplayer Bingo Game,” is a real-time, cross-platform gaming application consisting of a Flutter-based mobile front end and a Node.js/Express backend that communicates over both REST APIs and WebSocket connections using Socket.IO. The application allows two registered users to create or join a private game room using a shareable room code, receive a randomly generated 5×5 number grid, and take turns marking numbers in real time while the opposing player's screen updates instantly. The system implements line, column and diagonal win-detection logic, persists user identity and match statistics (wins, losses, games played) in MongoDB, supports authentication through both traditional email/password credentials and Google OAuth via Firebase, and includes a single-player “Bot” mode for practising without a second human player.

The report is organised into eight chapters covering the learning platform, the domain of the project, its practical application, the tools and technologies used, the specific tasks undertaken during development, the outcomes and skills gained, the broader benefits of the Course experience, and a concluding chapter that reflects on the overall journey. Screenshots, architecture descriptions, API and WebSocket event tables, and representative code excerpts from the actual project source (both the Flutter client and the Node.js backend) are included throughout to substantiate the technical depth of the work completed.

Overall, the Course enabled hands-on, project-driven mastery of Flutter/Dart for mobile UI development, Node.js/Express for REST API design, Socket.IO for real-time bidirectional communication, MongoDB for data persistence, and JWT/session-based authentication — skills directly relevant to the student's long-term goal of working as an Software Engineer.

TABLE OF CONTENTS

	Section	Page
	Certificate	i
	Acknowledgement	ii
	Abstract	iii
Chapter 1:	Introduction to the Learning Platform	1-2
Chapter 2:	Project Domain	3-4
Chapter 3:	Application of the Project	5-6
Chapter 4:	Tools, Technologies, and Skills Utilized	7-9
Chapter 5:	Tasks Undertaken / Project Contributions	10-17
Chapter 6:	Outcomes of the Project & Learning	18-19
Chapter 7:	Benefits of the Course Experience	20-21
Chapter 8:	Conclusion	22
	References	23
	Certificate of Course Completion	24

Chapter 1: Introduction to the Learning Platform (Udemy)

1.1 Overview of Course Program I

Course Program I, as prescribed under the Bachelor of Computer Applications (BCA) curriculum of The ICFAI University, Dehradun, requires every student to undertake a structured, industry-relevant learning and project experience during the summer break between the second and third years of study. While many students complete this requirement inside a company environment, the University also permits and encourages self-directed, outcome-based learning through recognised online platforms, provided the student demonstrates equivalent rigour: a fixed duration of study, a defined curriculum, hands-on assignments, and a substantial capstone project that can be evaluated exactly like a company-based Course deliverable. This report documents an Course of the second kind — a self-paced, structured learning engagement completed entirely on the Udemy platform, culminating in a full-stack, real-time multiplayer application built from first principles.

1.2 About Udemy as a Learning Organization

Udemy is one of the world's largest online learning marketplaces, hosting tens of thousands of courses across software development, data science, design, business, and other professional domains. Unlike a traditional classroom, Udemy operates on an instructor-marketplace model: subject-matter experts and working professionals design and publish self-contained video courses, which learners purchase or access and then complete at their own pace. For a student pursuing a technical Course, Udemy offers several characteristics that make it a credible substitute for, or complement to, a company-based Course:

- Industry-authored curriculum – courses are typically created and maintained by practising developers, so the material reflects current, real-world tooling and practices rather than purely academic content.
- Project-based pedagogy – most well-rated development courses are structured around building complete, working applications rather than isolated exercises, closely mirroring how a junior developer would be onboarded inside a company.

1.3 The Course Undertaken – “Flutter & Dart Programming”

For this Course, the course followed was “Flutter & Dart Programming,” an app-development-focused course on Udemy that teaches the Dart programming language from fundamentals through to building complete, production-style Flutter applications for Android, iOS, and the web from a single codebase. The course was chosen deliberately because Flutter's single-codebase, hot-reload development model, combined with a matching real-time backend, allows a solo student developer to build and iterate on a genuinely full-stack, cross-platform product within a single Course cycle — which is exactly what was achieved with the Multiplayer Bingo Game project described in this report.

The course curriculum broadly progressed through the following stages, each of which was directly applied to the Course project:

1. Dart language fundamentals – variables, null-safety, collections, asynchronous programming with Future/async-await, and object-oriented concepts (classes, inheritance) that underpin every Flutter widget.
2. Flutter widget fundamentals – the StatelessWidget/StatefulWidget model, layout widgets (Row, Column, Stack, GridView), and navigation.
3. State management and lifecycle – managing UI state across asynchronous events such as network responses and WebSocket callbacks.
4. Networking in Flutter – consuming REST APIs with the http package, handling JSON serialisation/deserialisation, and integrating third-party packages such as socket_io_client and shared_preferences from pub.dev.
5. Building and shipping a real application – project structuring app theming, launcher icon configuration, and preparing an Android build.

1.4 Why a Self-Paced Course Model Was an Appropriate Fit

Because the Course's objective was to acquire and demonstrate practical, job-ready app-development skills rather than to be assigned to a pre-existing company codebase, the Udemy self-paced model offered a distinct advantage: complete ownership of the project's architecture, scope, and technology choices from day one. Every design decision described later in this report — choosing Socket.IO over plain HTTP polling for real-time gameplay, choosing MongoDB for flexible player-statistics storage, mirroring the kind of ownership a junior engineer is expected to take on in a real product team, while still being anchored to the structured teaching sequence of the course.

Chapter 2: Project Domain

2.1 Domain Identification

The project undertaken during this Course falls within the domain of Software Development, specifically the sub-domain of Cross-Platform Mobile Application Development combined with Real-Time Backend Systems Engineering. This domain sits at the intersection of three well-established areas of computer science and software engineering:

- Mobile Application Development – designing and implementing a responsive, multi-screen user interface using the Flutter framework and the Dart language.
- Backend / Server-Side Engineering – designing REST APIs and persistent data models using Node.js, Express.js, and MongoDB.
- Real-Time Distributed Systems – implementing bidirectional, low-latency communication between multiple connected clients using the WebSocket protocol via Socket.IO, including state synchronisation, turn management, and disconnect/reconnect handling.

2.2 Responsibilities Within the Domain

As the sole developer on this Course project, responsibilities spanned the entire software development lifecycle rather than a single specialised role. This full-stack ownership is a deliberate and important aspect of the learning outcome: it required reasoning about a feature from the database schema, through the API and WebSocket contract, all the way to the on-screen widget that a player interacts with. The specific responsibilities undertaken included:

- Requirement definition – deciding what a “minimum lovable” multiplayer Bingo experience should include (room codes, live opponent view, win detection, match history).
- System design – defining the client-server architecture, the shape of the REST API, and the vocabulary of WebSocket events exchanged between client and server.
- Backend implementation – building the Express server, MongoDB data models, authentication routes, and the Socket.IO event handlers that drive gameplay.
- Frontend implementation – building every Flutter screen (landing, login, register, lobby, room creation/joining, live game grid, chat, and result screens) and the supporting service classes that talk to the backend.

- Testing and debugging – manually testing multi-device gameplay (two emulators / two physical devices joining the same room), fixing race conditions in turn management, and handling network drop scenarios.
- Documentation – writing a setup guide (FLUTTER_SETUP.md) describing the API, WebSocket events, environment configuration, and deployment steps for the backend, so that the project could be picked up, understood, or extended by another developer.

2.3 Relevance of the Domain to Industry

Real-time, cross-platform applications of the kind built in this project — multiplayer games, live chat, collaborative editors, ride-hailing trackers, live dashboards — represent a large and growing segment of modern software products. The specific technology combination used here (Flutter for a single cross-platform client, Node.js/Express for the API layer, Socket.IO for real-time transport, and MongoDB for flexible document storage) is a widely adopted, industry-standard stack, frequently referred to as part of the broader JavaScript/TypeScript-centric full-stack ecosystem. Gaining hands-on fluency in this stack directly supports the student's stated career direction of becoming an Agentic AI / full-stack software engineer, since most agentic and AI-driven products are ultimately delivered to end users through exactly this kind of client-server, API-and-event-driven architecture.

Chapter 3: Application of the Project

3.1 Purpose of the Project

The purpose of the Multiplayer Bingo Game project was to translate the theoretical concepts taught in the Udemy “Flutter & Dart Programming” course into a single, coherent, end-to-end application that could not be completed by following the course videos alone. Bingo was deliberately chosen as the game domain because, despite its simple rules, a faithful real-time implementation touches almost every hard problem in application engineering: user identity and authentication, room-based session management, state synchronization between two independent clients, race-condition-free turn enforcement, graceful handling of network disconnection, and persistence of long-term statistics — all of which had to be designed and solved independently rather than copied from a tutorial.

3.2 How the Project Was Applied in a Realistic Technical Context

The application was structured to resemble a real, shippable product rather than a classroom demo. Concretely, the following real-world product concerns were addressed:

- **Account system** – users can register and log in with an email and password, or sign in with Google via Firebase Authentication, matching the dual authentication pattern common in production consumer apps.
- **Session portability** – the backend issues a JWT token for mobile clients (30-day expiry) while simultaneously supporting cookie-based sessions (via express-session and connect-mongo) for any web client, so the same backend can serve the Flutter app, a future web client, or Postman-based testing without modification.
- **Room-based matchmaking** – rather than a global matchmaking queue, the game uses short, shareable alphanumeric room codes (generated server-side) that one player creates and shares with a friend, similar to how consumer game apps (e.g., Roblox, Among Us) implement casual private-lobby play.
- **Live, low-latency gameplay** – once both players are in a room, every number selection, turn change, and win event is pushed instantly over a persistent WebSocket connection rather than being polled, so both screens update within milliseconds of each other.
- **Resilience to disconnection** – if a player's socket disconnects (e.g., due to a flaky network), the server does not immediately end the match; it waits a short grace period (5 seconds) to allow the client to reconnect before declaring the remaining

player the winner, which mirrors how production real-time games avoid unfairly penalising brief network blips.

- **Single-player practice mode** – A Bot opponent (implemented as a virtual player object identified by role: "Bot") lets a user test and practice game mechanics without needing a second human player, useful both for the student's own testing and for a real end user with no one to play against.

3.3 Benefit and Relevance to a Real Organization

Although this project was completed independently rather than inside a hiring company, its architecture and code quality were deliberately built to the standard expected of a real product team, so that the resulting skillset and portfolio artefact are directly transferable to a professional engineering role. In an organisational context, a project of this shape would typically deliver value as: (a) a demonstrably working real-time product feature that a company could ship to end users, (b) a reusable backend pattern (JWT + session hybrid authentication, Socket.IO room management) applicable to other real-time features such as live chat or collaborative tools, and (c) a well-documented codebase (see FLUTTER_SETUP.md, described in Chapter 5) that reduces onboarding time for any engineer who needs to extend it.

Chapter 4: Tools, Technologies, and Skills Utilised

4.1 Technology Stack Summary

The project was built as two independently deployable services — a Flutter mobile client and a Node.js backend — that communicate over REST and WebSocket protocols. The table below summarises the primary technologies used in each layer.

Layer	Technology / Package	Purpose
Mobile Client	Flutter (Dart, SDK ^3.10.4)	Cross-platform UI framework for Android, iOS and web from one codebase
Mobile Client	http (^1.2.1)	REST API calls to the backend (register, login, room create/join)
Mobile Client	socket_io_client (^3.0.2)	Real-time WebSocket communication with the backend during gameplay
Mobile Client	shared_preferences (^2.2.3)	Local persistence of the JWT token and lightweight user session data
Mobile Client	flutter_launcher_icons (^0.14.4)	Generating the Android adaptive app icon from a single source image
Backend	Node.js + Express (^5.2.1)	HTTP server and REST API routing
Backend	Socket.IO (^4.8.3)	Bidirectional, event-based real-time communication layer
Backend	MongoDB(^9.1.4)	Document database and ODM for persisting user accounts and statistics
Backend	jsonwebtoken (^9.0.3)	Issuing and verifying JWTs for mobile-client authentication
Backend	express-session + connect-mongo (^6.0.0)	Cookie-based session persistence in MongoDB for web clients
Backend	firebase-admin (^13.6.0)	Verifying Google OAuth ID tokens for “Sign in with Google”
Backend	cors (^2.8.5)	Configurable cross-origin resource sharing for web and mobile origins
Backend	dotenv (^17.4.2)	Loading environment-specific configuration (.env) into the server

4.2 Development Tools and Environment

- **Udemy** – primary learning platform for the “Flutter & Dart Programming” course
- **Visual Studio Code** – primary IDE for both the Flutter/Dart client and the Node.js backend
- **Android Emulator / physical Android device** – for running and hot-reloading the Flutter app during development

- **Postman** – for manually testing REST endpoints (register, login, room creation) independent of the mobile UI, as documented in FLUTTER_SETUP.md
- **MongoDB Atlas** – cloud-hosted MongoDB cluster used as the persistent data store (referenced via the MONGODB_URI environment variable)
- **Git and GitHub** – version control for both the app_zip (Flutter client) and backend_zip (Node.js server) codebases
- **Nodemon** – auto-restarting the backend server on file changes during development (npm run dev)
- **Docker** – containerising the backend for consistent deployment, with a Dockerfile based on the official node:18 image
- **Render** – recommended cloud host for deploying the persistent, WebSocket-friendly Node.js backend

4.3 Technical Skills Developed

4.3.1 Mobile / Frontend Skills

- Structuring a multi-screen Flutter application using a clear separation of pages, services, utilities, theming, and configuration (lib/pages, lib/services, lib/utils, lib/theme, lib/config)
- Managing asynchronous UI state driven by both REST responses (Future/async-await) and continuous WebSocket event streams
- Building a custom 5×5 interactive grid widget with per-cell selection state, distinct from a generic Grid View pattern
- Implementing an in-app real-time chat widget (chat_box.dart) alongside the core game screen
- Configuring environment-specific backend URLs at build time using --dart-define, and packaging the app with a custom launcher icon

4.3.2 Backend Skills

- Designing a REST API with Express 5 covering authentication (register, login, Google OAuth, token verification) and game-room lifecycle (create, join)
- Implementing a hybrid authentication scheme — JWT bearer tokens for mobile clients and cookie-backed sessions (stored in MongoDB via connect-mongo) for web clients — selected dynamically per request using an x-client-type header
- Modelling persistent data with Mongoose schemas, including unique-index-based constraints (unique email) and derived statistics fields (gamesPlayed, win, loss, draw)

- Building an in-memory room-management layer (a JavaScript Map keyed by room ID) to track connected players, host role, turn order, and match state without needing a dedicated database write on every move
- Implementing Web-Socket event handlers with Socket.IO for room joining, turn-based number selection, win declaration, chat relay, explicit leave, and implicit disconnect handling with a reconnection grace window
- Configuring environment-aware CORS policy that behaves permissively in development but enforces a strict allow-list of origins in production
- Verifying Google Sign-In tokens server-side using the Firebase Admin SDK, and auto-provisioning a MongoDB user record on first Google login

4.3.3 Cross-Cutting / Professional Skills

- Independent architectural decision-making — choosing Web-Sockets over polling, choosing a hybrid JWT/session auth model, and choosing an in-memory room store over a database-backed one for latency reasons
- Writing developer-facing technical documentation (FLUTTER_SETUP.md) covering API contracts, Web-Socket event contracts, environment variables, deployment options and troubleshooting steps
- Debugging distributed, asynchronous systems where a bug can only be reproduced by running two clients simultaneously against one shared server
- Time management and self-discipline required to complete a self-paced course and an original capstone project within a fixed Course window without external supervision

Chapter 5: Tasks Undertaken / Project Contributions

5.1 Project Title and Objective

Project Title: Multiplayer Bingo-Game – A Real-Time Cross-Platform Gaming Application.

Objective: To design and independently develop a complete, real-time, two-player Bingo game consisting of a Flutter mobile client and a Node.js/Express/Socket.IO backend, incorporating user authentication, private room-based matchmaking, synchronized live gameplay, in-game chat, persistent player statistics, and a single-player Bot mode, as a practical, portfolio-grade application of the skills taught in the Udemy “Flutter & Dart Programming” course.

5.2 System Architecture

The system follows a classic three-part client-server architecture, with the backend exposing two parallel communication channels to the client: a stateless REST API for authentication and room bootstrapping, and a stateful, persistent Web-Socket channel (Socket.IO) for everything that happens once a match is live. This separation keeps one-time operations (like registering an account or creating a room) simple and cacheable, while keeping high-frequency, low-latency operations (like marking a number on the grid) on a channel built for exactly that purpose.

5.3 Project Folder Structure

Both codebases were organised using a clear, layered folder structure to keep concerns separated — a practice adopted directly from the project-based teaching style of the Udemy course and reinforced through independent research into standard Node.js/Flutter project conventions.

Codebase	Key Folders / Files	Responsibility
Flutter Client	lib/pages/	Screen-level widgets: landing, login, register, set_password, lobby, create_room, join_room, create_bot, game_home, grid, game, result
Flutter Client	lib/services/	api_service.dart (REST calls), socket_service.dart (Socket.IO client wrapper)
Flutter Client	lib/utills/	bingo_game.dart – pure game logic: grid generation, marking numbers, win detection, scoring
Flutter Client	lib/theme/, lib/config/	App-wide theming (app_theme.dart) and environment configuration (app_config.dart)
Backend	src/server.js	Express app bootstrap, middleware, CORS, session store, Socket.IO server initialisation
Backend	src/routes/	auth.route.js (authentication), game.route.js (room creation/joining + in-memory room store)
Backend	src/config/	db.js (MongoDB connection), socket.js (all Socket.IO event handlers)
Backend	src/database/	User.js – Mongoose schema for user accounts and statistics

5.4 REST API Design and Implementation

The backend exposes a small, focused REST surface. Authentication routes are mounted at `/api`, and room routes are mounted at `/api/game`. All endpoints return JSON, and authenticated endpoints accept either an `Authorization: Bearer <JWT>` header (mobile clients) or an active session cookie (web clients), selected using a custom `x-client-type` request header read inside the middleware.

Method	Endpoint	Description
POST	<code>/api/register</code>	Creates a new user with name, email and a bcrypt-style hashed password; returns a signed JWT
POST	<code>/api/login</code>	Validates email/password credentials against the User collection and returns a JWT (or sets a session cookie for web clients)
POST	<code>/api/google</code>	Verifies a Google ID token via firebase-admin, auto-creates a User record on first login (provider: "google"), and returns a JWT/session
GET	<code>/api/verify-token</code>	Validates an existing JWT/session and returns the current authenticated user's profile
POST	<code>/api/game/room/create</code>	Generates a short random room ID (<code>Math.random().toString(36)</code>) and registers a new room entry in the in-memory rooms Map
POST	<code>/api/game/room/join</code>	Validates that a given roomId exists and is not already full (max 2 players) before the client opens a WebSocket connection to it
GET	<code>/health</code>	Lightweight health-check endpoint used for uptime monitoring / container orchestration

5.4.1 Representative Backend Code – User Schema (`database/User.js`)

```
const userSchema = new mongoose.Schema({
  {
    name: String,
    email: { type: String, unique: true },
    password: { type: String, default: null },
    provider: { type: String, default: "local" },

    gamesPlayed: { type: Number, default: 0 },
    win: { type: Number, default: 0 },
    loss: { type: Number, default: 0 },
    draw: { type: Number, default: 0 },
    lastLogin: { type: Date, default: null },
  },
  { timestamps: true },
});
export default mongoose.model("User", userSchema);
```

This schema was deliberately kept minimal but includes a provider field to distinguish local (email/password) accounts from Google-authenticated accounts, and four running statistics counters (gamesPlayed, win, loss, draw) that are incremented directly inside the WebSocket win-handler rather than being recomputed from a match-history collection — a pragmatic trade-off chosen for simplicity given the project's scope.

5.4.2 Representative Backend Code – Room Creation (routes/game.route.js)

```
const rooms = new Map();
const genRoomId = () => Math.random().toString(36).substring(2, 9);

router.post("/room/create", (req, res) => {
  const roomId = genRoomId();
  rooms.set(roomId, {
    roomId, players: [], hostUserId: null,
    grids: {}, role: null, turnIndex: 0, started: false,
  });
  res.json({ success: true, roomId });
});
```

Rooms are intentionally stored in a server-side in-memory JavaScript Map rather than in MongoDB. Because a room's lifetime is short (the duration of a single match) and its state changes on every move, keeping it in memory avoids a database round-trip on every number selection, which would otherwise add unnecessary latency to the real-time gameplay loop. The trade-off — room state is lost if the server restarts — was judged acceptable for a game of this scope and is documented as a known limitation in FLUTTER_SETUP.md.

5.5 Real-Time Gameplay: Socket.IO Event Contract

The heart of the project is the WebSocket event contract implemented in src/config/socket.js and mirrored on the client in lib/services/socket_service.dart. The table below lists every event exchanged between the client and the server during a match.

Direction	Event Name	Payload / Purpose
Client → Server	join-room	{ roomId, user } – adds/updates the caller as a player in the given room (max 2 real players; a Bot is added as a synthetic player)
Server → Client	room-joined	players[] – broadcast to everyone in the room whenever the player list changes

Direction	Event Name	Payload / Purpose
Client → Server	start-game	{ roomId } – sent once both players are present; server marks the room started and selects the first turn (Host goes first)
Server → Client	game-start	{ turnUserId } – tells both clients whose turn it is at kickoff
Client → Server	game:select-number	{ roomId, number, userId } – sent when the player whose turn it is taps a cell; server rejects the call if it is not that user's turn
Server → Client	game:update	{ number } – broadcasts the selected number to both clients so both grids mark it simultaneously
Server → Client	game:turn	{ userId } – broadcasts whose turn is next after a valid selection
Client → Server	game:win	{ roomId, userId } – sent by the client that locally detects a completed win (5 lines/rows/diagonals)
Server → Client	game:win	{ userId } – confirmed/broadcast to both clients; server then updates win/loss statistics in MongoDB and schedules room cleanup
Client ↔ Server	chat:send / chat:receive	{ roomId, message, senderId, senderName } – relays in-match chat messages, timestamped server-side
Client → Server	leave-room	{ roomId } – explicit exit; server removes the player and deletes the room if it becomes empty
(implicit)	disconnect	Server detects a dropped socket, waits a 5-second grace period for reconnection before removing the player and, if a match was in progress, declaring the remaining player the winner

5.5.1 Representative Backend Code – Turn-Enforced Number Selection

```
socket.on("game:select-number", ({ roomId, number, userId }) => {
  const room = rooms.get(roomId);
  if (!room || room.turnUserId !== userId) return; // reject out-of-turn moves

  io.to(roomId).emit("game:update", { number });
  if (room.players.length < 2) return;

  const nextPlayer = getNextPlayer(room.players, userId);
  if (!nextPlayer) return;

  room.turnUserId = nextPlayer.userId;
  io.to(roomId).emit("game:turn", { userId: room.turnUserId });
});
```

This handler is the core of the game's fairness guarantee: every incoming `game:select-number` event is checked against `room.turnUserId` before it is broadcast, so a malicious or buggy client cannot play out of turn — the authoritative turn state lives on the server, not on either client.

5.5.2 Representative Backend Code – Disconnect / Reconnection Grace Period

```
socket.on("disconnect", async () => {
  for (const [roomId, room] of rooms.entries()) {
    const player = room.players.find((p) => p.socketId === socket.id);
    if (!player) continue;

    setTimeout(async () => {
      const stillExists = room.players.find((p) => p.userId === player.userId);
      if (stillExists && stillExists.socketId !== socket.id) return; // reconnected in
time

      room.players = room.players.filter((p) => p.userId !== player.userId);
      io.to(roomId).emit("room-joined", room.players);

      if (room.started && room.players.length === 1) {
        const winnerId = room.players[0].userId;
        io.to(roomId).emit("game:win", { userId: winnerId });
        await updateStatsOnResult(winnerId, player.userId);
        rooms.delete(roomId);
      }
    }, 5000); // 5-second grace period before treating the drop as final
  }
});
```

This was one of the more subtle problems solved independently during the Course: a naive implementation would remove a player — and potentially end the match — the instant their socket disconnected, which unfairly punishes players who briefly lose signal (e.g., switching from Wi-Fi to mobile data). The chosen solution defers the removal by five seconds and checks whether a new socket for the same `userId` has since re-joined the room, only finalising the disconnect (and, if a match was live, awarding the win to the opponent) if the player has not returned in time.

5.6 Frontend (Flutter) Implementation

On the client side, the application is organised as a sequence of screens that mirror the natural flow of a multiplayer match: from account access, to lobby, to room creation/joining, to live gameplay, to a final result screen.

Screen (lib/pages/)	Approx. Lines	Role
landing.dart	195	App entry / splash screen with branding and navigation into login or registration
login.dart / register.dart	137 / 118	Email/password and Google Sign-In authentication forms, calling <code>api_service.dart</code>
set_password.dart	82	Password-set/reset flow for account security
lobby.dart	276	Post-login hub showing the player's stats and options to create or join a room

Screen (lib/pages/)	Approx. Lines	Role
create_room.dart / join_room.dart	178 / 139	Room creation (calls /api/game/room/create) and room joining by code (calls /api/game/room/join, then opens the socket)
create_bot.dart	68	Starts a single-player match against a synthetic Bot opponent for practice
game_home.dart	126	Pre-game waiting screen showing connected players before the match starts
grid.dart	258	Renders the interactive 5×5 number grid and handles per-cell tap events
game.dart	451	Main live-match screen: orchestrates socket events, turn indicator, opponent grid view, and the in-game chat widget
result.dart	247	Post-match summary screen showing the winner, updated statistics, and a rematch/return-to-lobby option

5.6.1 Representative Client Code – Win Detection (*utils/bingo_game.dart*)

```
static bool _checkWin(List<List<Map<String, dynamic>>> grid) {
  int completedLines = 0;

  for (int r = 0; r < 5; r++) {
    // rows
    if (grid[r].every((cell) => cell['chosen'] == true)) completedLines++;
  }
  for (int c = 0; c < 5; c++) {
    // columns
    bool full = true;
    for (int r = 0; r < 5; r++) {
      if (grid[r][c]['chosen'] != true) { full = false; break; }
    }
    if (full) completedLines++;
  }
  // + both diagonals checked the same way
  return completedLines >= 5;
}
```

The win rule implemented is a variant of standard Bingo: instead of stopping at the first completed line, the game requires five completed lines in total across rows, columns and diagonals (a maximum of 12 possible lines on a 5×5 grid), which was a deliberate design choice made to lengthen and balance matches for a two-player, turn-based context, rather than reproducing the traditional single-line “full house” rule found in classic Bingo.

5.7 Authentication and Security Measures Implemented

- Passwords are never stored in plain text; the register endpoint hashes the password before persisting the User document.
- JWTs issued at login/register carry a 30-day expiry and are validated on every protected request via the /api/verify-token endpoint.

- Google Sign-In tokens are verified server-side against Firebase using firebase-admin rather than trusting any user data sent directly from the client.
- Session cookies (for web clients) are marked httpOnly and, in production, secure and sameSite: "none" to support cross-origin deployment between a separately hosted frontend and backend.
- CORS is environment-aware: permissive for localhost during development, but restricted to an explicit allow-list of origins (read from the CORS_ORIGINS environment variable) in production.

5.8 Deployment and DevOps Tasks

Beyond writing application code, the Course also involved packaging the backend for deployment, since a real-time Socket.IO server has different hosting requirements from a typical stateless REST API (it needs a platform that supports long-lived, persistent connections rather than short-lived serverless functions). The following deployment-related tasks were completed:

- Wrote a Dockerfile (based on node:18) so the backend can be built and run identically in any container-based environment.
- Documented, in FLUTTER_SETUP.md, why a serverless platform such as Vercel is not suitable for the Socket.IO server (serverless functions do not keep a persistent WebSocket connection alive), and recommended Render as a suitable always-on host instead.
- Externalised all secrets and environment-specific values (MONGODB_URI, JWT secret, SESSION_SECRET, CORS_ORIGINS, Firebase service-account credentials, PORT) into a .env file loaded via dotenv, keeping credentials out of source control.
- Configured the Flutter build to accept the backend's base URL via --dart-define, so the same client code can point at a local development server or a deployed production server without code changes.
- Generated a custom Android adaptive launcher icon using the flutter_launcher_icons package, giving the app a polished, installable look.

Chapter 6: Outcomes of the Project & Learning

6.1 Project Outcomes

By the end of the Course period, the following concrete, working deliverables had been produced:

6. A fully functional Flutter mobile client (Android build verified) covering the complete user journey: landing → register/login (including Google Sign-In) → lobby → create/join room → live match with real-time grid updates and in-game chat → result screen with updated statistics.
7. A production-style Node.js/Express backend exposing a documented REST API and a full Socket.IO real-time event contract, backed by MongoDB for persistent user data.
8. A working single-player Bot mode, allowing the game to be demonstrated and tested without needing two simultaneous human testers.
9. Developer documentation (FLUTTER_SETUP.md) describing environment configuration, the REST API, the WebSocket event contract, and deployment guidance — written so the project could be handed off to, or extended by, another developer.
10. A Dockerised, deployment-ready backend configuration, with an explicit, reasoned recommendation for a WebSocket-compatible hosting platform (Render) over a serverless one (Vercel) for the server component.

6.2 Technical Skills Gained

- Practical fluency in Dart and the Flutter widget/state model, gained by building eleven distinct, interconnected screens rather than isolated course exercises.
- Confident, independent use of Socket.IO to design a bidirectional event contract from scratch, including turn enforcement, broadcast-to-room semantics, and graceful disconnect handling.
- Practical experience designing a hybrid authentication system (JWT + session-based) that serves two different client types (mobile and web) from a single backend.
- Hands-on schema design with Mongoose/MongoDB, including unique constraints and incrementally updated statistics fields.
- Experience reasoning about state consistency and race conditions in a distributed, multi-client, real-time system — a class of problem rarely encountered in single-user CRUD applications.

- Working knowledge of environment-based configuration, CORS policy design, and containerization (Docker) for deployment.

6.3 Soft Skills and Professional Learning

- Self-directed learning – successfully following a structured Udemy curriculum without a classroom or company supervisor, and translating course concepts into an original, larger-scope project.
- Independent problem-solving – diagnosing and fixing issues (such as out-of-turn moves and unfair disconnect penalties) that had no direct answer in the course material and required original design thinking.
- Technical documentation – writing FLUTTER_SETUP.md forced a habit of explaining design decisions clearly enough for another developer (or a future version of the student) to understand and extend the system.
- Project planning and time management – balancing structured course consumption with independent project development within a fixed Course timeline.
- Resilience and debugging discipline – multi-client, real-time bugs are hard to reproduce; the project built patience and a systematic approach to isolating issues (e.g., logging on both client and server, testing with two devices side by side).

6.4 Mapping of Learning Outcomes to the Course Curriculum

Udemy Course Topic	Applied In Project As
Dart fundamentals & async/await	All service calls in api_service.dart and every socket callback in socket_service.dart
Flutter widgets & navigation	Eleven-screen navigation flow from landing.dart through to result.dart
State management across async events	game.dart orchestrating simultaneous REST, socket and local grid state
Consuming REST APIs / packages from pub.dev	http, socket_io_client and shared_preferences integration
Building and packaging a real app	flutter_launcher_icons configuration and --dart-define based environment builds

Chapter 7: Benefits of the Course Experience

7.1 Academic Benefits

The Course provided a bridge between the theoretical, syllabus-driven content of the BCA programme and the practical realities of building and shipping a working software product. Concepts such as client-server architecture, API design, and database schema design — typically introduced in isolation across separate subjects — had to be combined and reasoned about together in a single, coherent system, deepening conceptual understanding far beyond what classroom examples alone typically provide.

7.2 Career and Skill-Building Benefits

The technology stack used in this project — Flutter, Node.js, Express, Socket.IO, MongoDB, JWT-based authentication — is directly aligned with the skillset expected of an entry-level full-stack or mobile developer, and forms a strong practical foundation for the student's longer-term goal of specialising in Agentic AI engineering, where products are typically delivered through similar client-server, API-and-event-driven architectures with a conversational or agentic interface layered on top. Building this project independently — rather than only following along with course videos — also produced a genuine, demonstrable portfolio artefact that can be shown to prospective employers or included in a resume/GitHub profile, which is more persuasive evidence of ability than a course-completion certificate alone.

7.3 Practical / Real-World Problem-Solving Benefits

- Learned to design for imperfect networks (the disconnect grace-period logic) rather than only for the “happy path,” a mindset essential in real production systems.
- Learned to make and justify architectural trade-offs (e.g., in-memory room state vs. database-backed room state) under real constraints of latency and simplicity, rather than being told the “correct” answer.
- Gained first-hand appreciation of why real-time systems need a different transport (WebSockets) than typical request-response APIs, by having personally built both kinds of endpoint in the same project.
- Learned to write documentation intended for another engineer, which is a distinct skill from writing code that merely works, and is directly transferable to collaborative team environments.

7.4 Personal Development Benefits

On a personal level, the Course strengthened self-discipline, time management, and confidence in tackling an ambiguous, open-ended problem without a fixed specification — the entire scope of the Bingo game, from its win rule to its authentication strategy, had to be decided by the student rather than handed down by an employer or instructor. This experience of full ownership, from the first line of code to a deployable, documented product, is considered one of the most valuable outcomes of the Course.

Chapter 8: Conclusion

Course Program I, completed through structured self-paced learning on Udemy via the “Flutter & Dart Programming” course, culminated in the design and development of a complete, real-time, cross-platform Multiplayer Bingo Game. What began as a set of course lectures on Dart syntax and Flutter widgets was extended, through independent design and engineering effort, into a full-stack product encompassing a Flutter mobile client, a Node.js/Express REST API, a Socket.IO real-time communication layer, MongoDB-backed persistence, and dual-mode (JWT and session-based) authentication with both traditional and Google OAuth login.

The project required solving problems that go well beyond following a tutorial: enforcing server-authoritative turn order to prevent cheating, gracefully tolerating brief network disconnections without unfairly ending a match, designing a room-based matchmaking flow using shareable codes, and documenting the system clearly enough for another developer to pick up and extend. In working through these problems independently, the Course delivered exactly the kind of applied, self-directed learning experience that Course Program I is intended to provide — comparable in rigour and outcome to a company-based Course, while allowing complete ownership over the technical direction of the project.

In summary, this Course successfully combined a recognised, industry-relevant online curriculum with an original, substantial software engineering project, resulting in a working application, a documented codebase, and a measurably expanded set of technical and professional skills.

References

- Udemy – “Flutter & Dart Programming” course, <https://www.udemy.com>
- Flutter Official Documentation, <https://docs.flutter.dev>
- Dart Language Tour, <https://dart.dev/guides>
- Node.js Official Documentation, <https://nodejs.org/en/docs>
- Express.js Official Guide, <https://expressjs.com>
- Socket.IO Official Documentation, <https://socket.io/docs/v4>
- MongoDB Official Documentation, <https://www.mongodb.com/docs>
- jsonwebtoken (npm package) Documentation, <https://www.npmjs.com/package/jsonwebtoken>
- Firebase Admin SDK Documentation, <https://firebase.google.com/docs/admin/setup>
- pub.dev – Dart/Flutter package registry, <https://pub.dev>
- Render – Cloud Application Hosting, <https://render.com/docs>
- Docker Official Documentation, <https://docs.docker.com>
- Project source repositories: Flutter client and Node.js backend, student's own implementation, 2026

Certificate of Course Completion



Certificate no: UC-5782335d-45b8-4950-b793-c93d48c4ead0
Certificate url: ude.my/UC-5782335d-45b8-4950-b793-c93d48c4ead0
Reference Number: 0004

CERTIFICATE OF COMPLETION

Flutter & Dart - The Complete Guide

Instructors **Academind by Maximilian Schwarzmüller, Maximilian Schwarzmüller**

Aditya Jaiswal

Date **May 25, 2026**

Length **30 total hours**